> 4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.
>
> Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
venkatesh@LAPTOP-LL9RQ023:~$ nano week4.c
venkatesh@LAPTOP-LL9RQ023:~$ cat week4.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t p1, p2;

    p1 = fork();

    if (p1 == 0) {
        printf("Child 1: PID=%d
", getpid());
        sleep(2);
        exit(1);
    }

    p2 = fork();

    if (p2 == 0) {
        printf("Child 2: PID=%d
", getpid());
        sleep(1);
        exit(2);
    }

    int status1, status2;

    waitpid(p1, &status1, 0);
    printf("Parent: Child 1 (PID=%d) finished with exit code %d
",
           p1, WEXITSTATUS(status1));

    waitpid(p2, &status2, 0);
    printf("Parent: Child 2 (PID=%d) finished with exit code %d
",
           p2, WEXITSTATUS(status2));
```

```
    return 0;
}
venkatesh@LAPTOP-LL9RQ023:~$ gcc week4.c
venkatesh@LAPTOP-LL9RQ023:~$ ./a.out
Child 2: PID=795
Child 1: PID=794
Parent: Child 1 (PID=794) finished with exit code 1
Parent: Child 2 (PID=795) finished with exit code 2
```